



Embeddings and vector stores

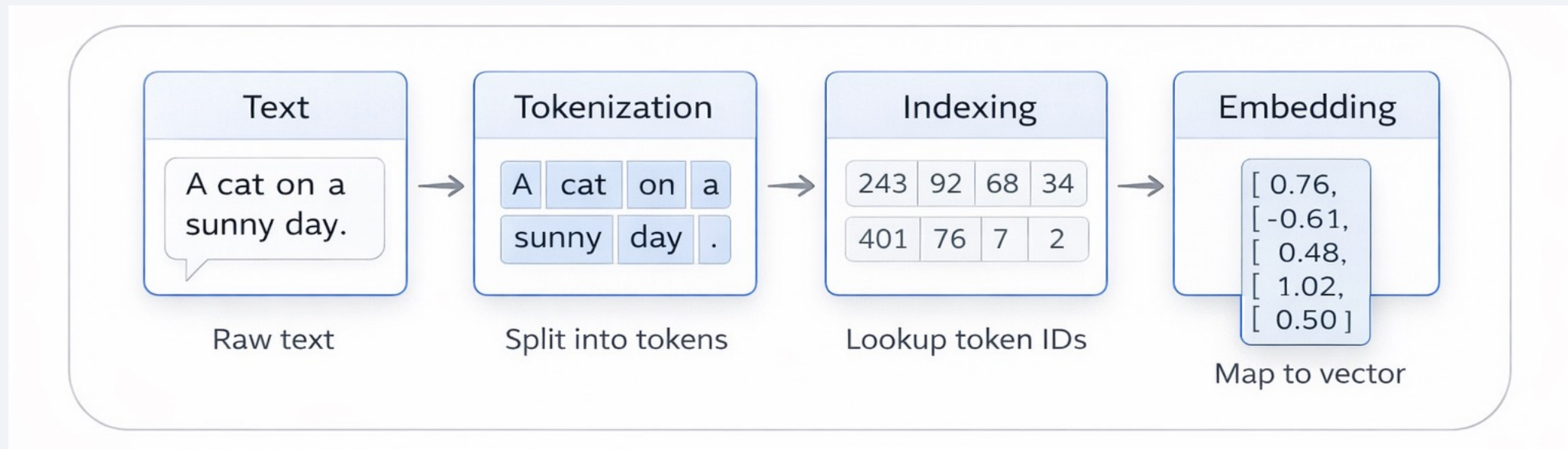
- Dr Maria Prokofieva

What we will cover

- Embeddings
- Types of embeddings and uses
- Semantic search
- Vector databases



From Text to Embeddings



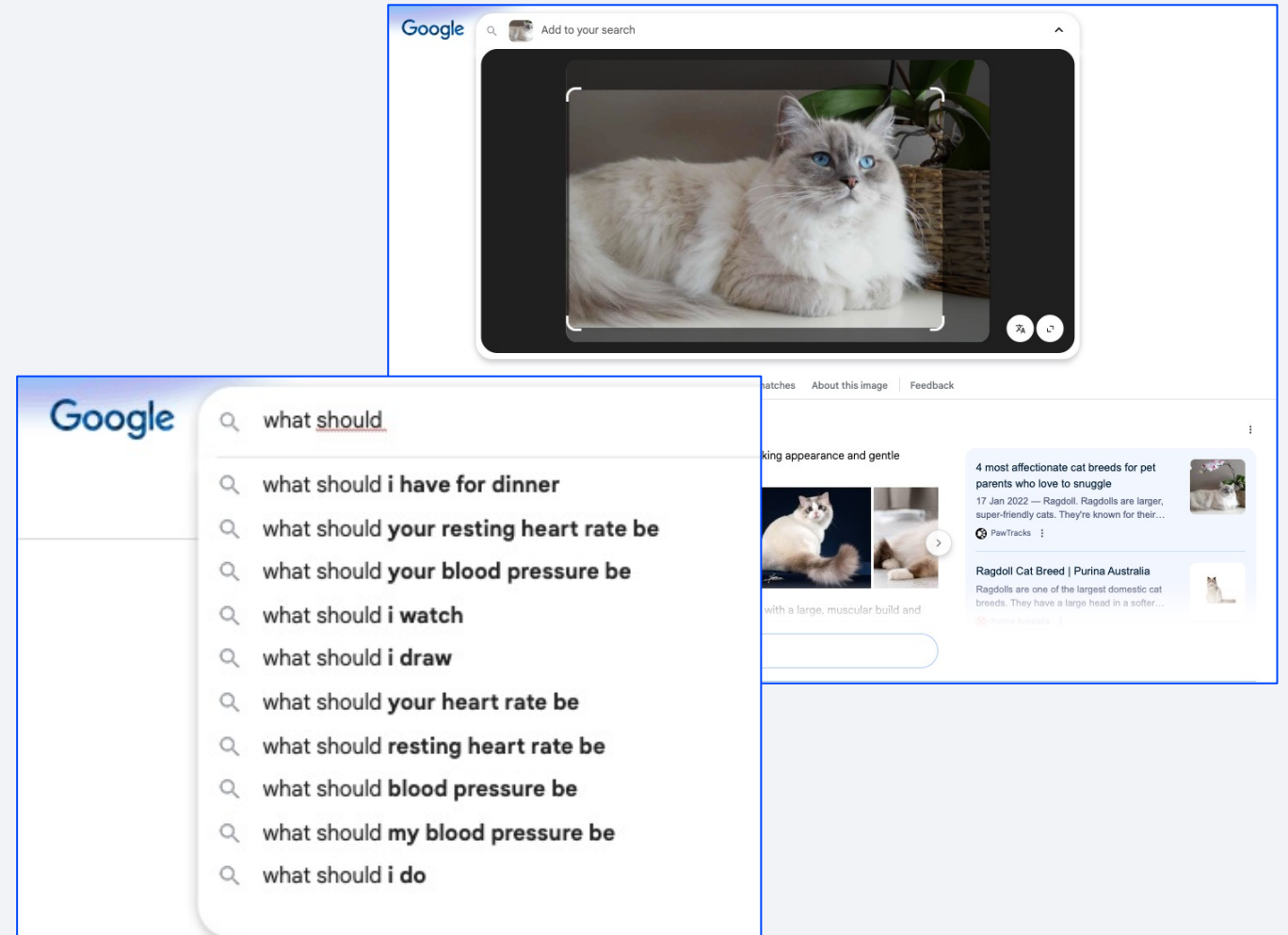
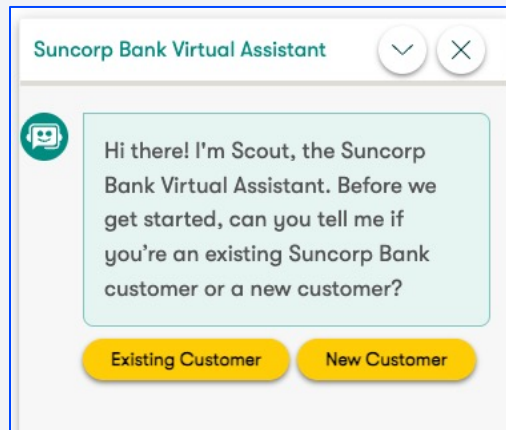
Embeddings

Semantic search

Recommendation engines

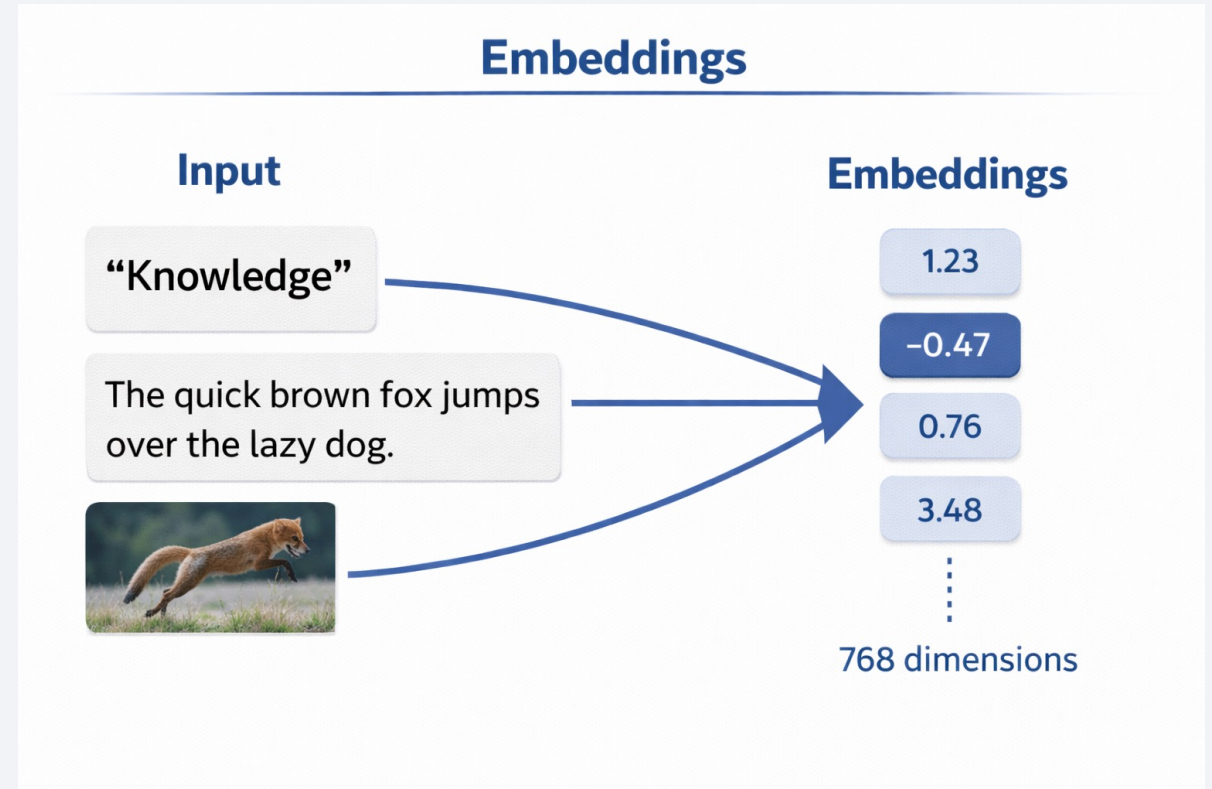
RAG

Image similarity and MANY MORE



What are Embeddings?

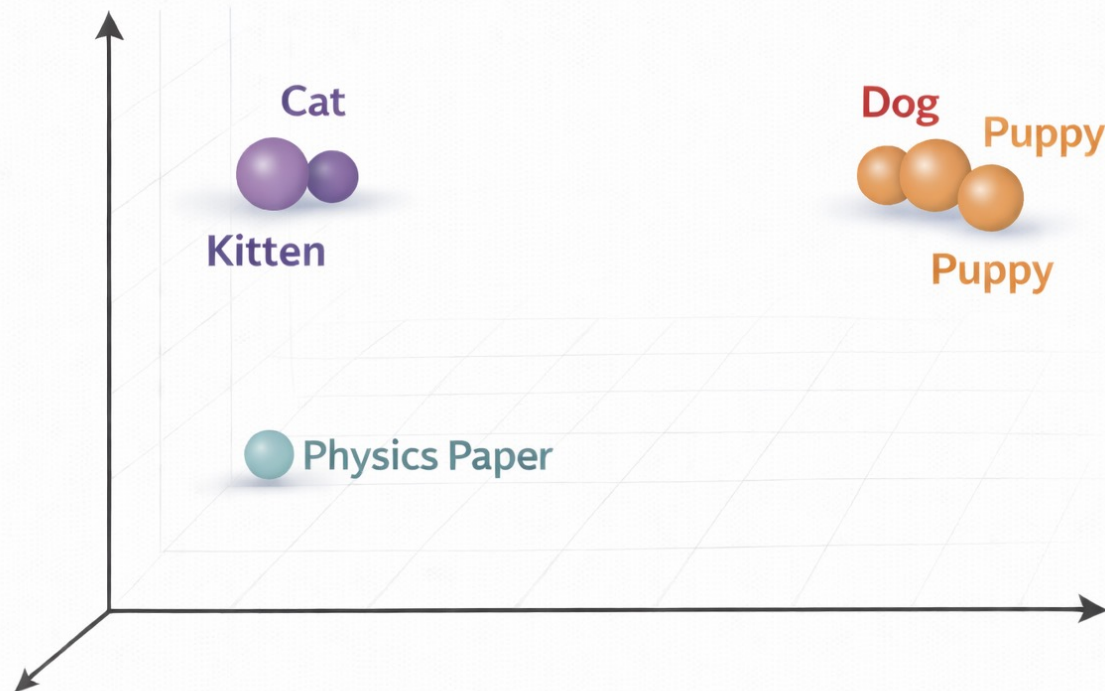
- Embeddings are low-dimensional numerical representations (usually dense vectors) of real-world data



Embeddings

Vector Space of Embeddings

Geometric distance approximates semantic / functional similarity



Embeddings

Why They Matter ?



Dimensionality reduction — from millions of sparse dimensions to hundreds/thousands of dense ones



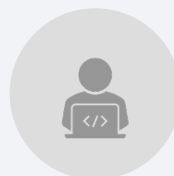
Learned similarity — cosine similarity or dot product becomes proxy for meaning



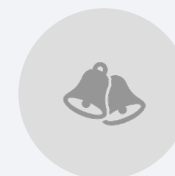
Arithmetic properties (especially in good static embeddings) — king – man + woman $\approx$ queen



Transferability — train once on huge corpus → reuse everywhere (search, classification, clustering, RAG, recommendation)



Enables distance-based algorithms — nearest-neighbor search, clustering, anomaly detection, retrieval-augmented generation (RAG)



Handles out-of-vocabulary gracefully with subword BPE + embeddings

Types of embeddings

Text Embeddings

Text

BERT, E5, OpenAI

RAG, search

Trained only on text corpora

Fluffy cat

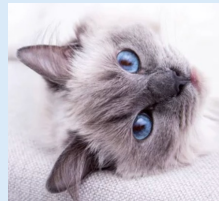
Image Embeddings

Images

DINOv2, CLIP

Vision search

Capture visual semantics
(objects, scenes, style, layout);
usually paired with text in
multimodal setups



Multimodal embeddings

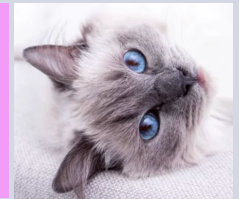
Images/Video + Text

CLIP, LLaVA

Cross-modal retrieval

Contrastive training with inter-
& intra-modality
positives/negatives

Fluffy cat



Embeddings by Granularity

Token

Subword token

Vector per token

Transformer
embedding layer

Fine-grained syntax,
morphology

No standalone
meaning

MLM pretraining,
NER, POS

Word

Full word

Vector per word

Word2Vec / GloVe
or contextual token

Lexical similarity

Weak for phrases,
polysemy

Legacy IR, word
similarity

Sentence

Sentence / short
text

One vector

Pooling over tokens
(SBERT, E5)

Captures sentence
meaning

Loses document
context

STS, short-query
retrieval

Document

Paragraph /
document

One vector

Pooling + long-
context models

Global topic,
discourse

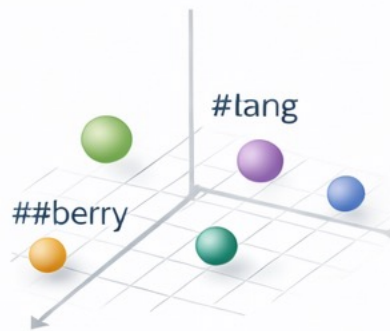
Chunking +
aggregation needed

RAG, doc retrieval,
rec sys

Embeddings by Granularity

Token
Embeddings

Vector space



Vector space

Word
Embeddings

Vector space



Vector space

Sentence
Embeddings

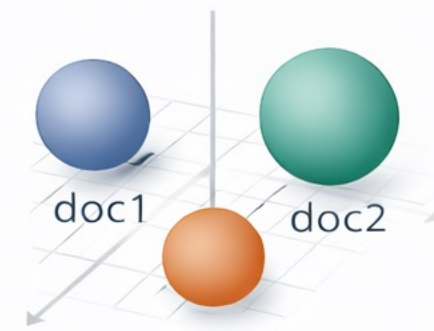
Vector space



Vector space

Document
Embeddings

Vector space

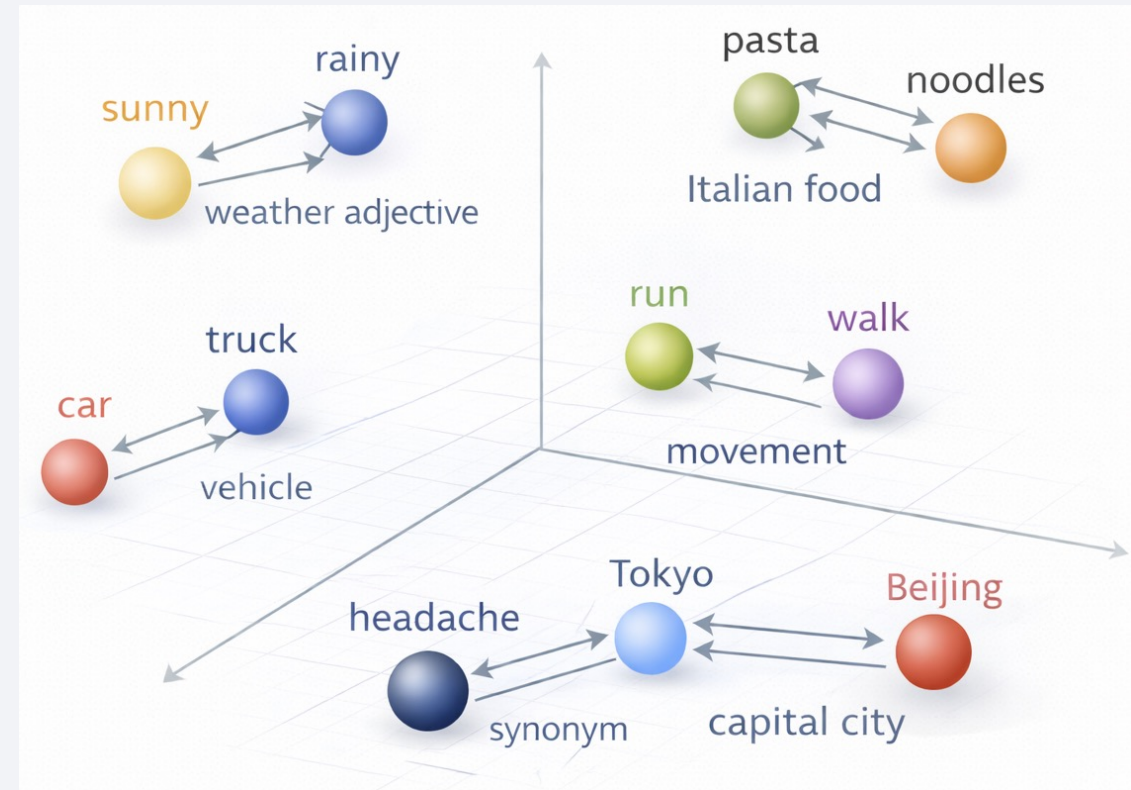


Vector space

Word Embeddings

Word embeddings represent words as **dense vectors** in a continuous space, where **distance and direction encode meaning**.

- Instead of treating words as IDs, embeddings place them so that:
- similar words are **close**
- relationships are **geometric**



Word Embeddings

Earlier representations:

- One-hot / Bag-of-Words / TF-IDF
→ sparse, high-dimensional, no semantics
- Word embeddings:
Dense (50–300 dims)
- Capture semantic similarity
- Enable analogy reasoning

<i>s p a r s e</i>							DENSE						
0	7	0	0	0	0	6	0	7	0	0	0	0	6
0	7	6	3	0	4	0	0	7	6	3	0	4	0
0	4	3	0	0	0	0	0	4	3	0	0	0	0
4	2	0	0	0	0	0	4	2	0	0	0	0	0
0	0	0	0	3	2	4	0	0	0	0	3	2	4

Word Embeddings

Static Word Embeddings (Classic Models)

Each word gets **one fixed vector**, regardless of context.

- **Word2Vec (2013)**
Learns from local context windows
→ strong analogies, fast
- **GloVe (2014)**
Learns from global co-occurrence statistics
→ strong semantic structure
- **fastText (2016)**
Uses character n-grams
→ handles morphology & unseen words

Limitation:

Polysemy is averaged
→ “bank” (river) = “bank” (finance)

A word/token’s vector is **context-dependent**
→ same token gets different embeddings in different sentences.

Tokenize → Transformer layers → hidden states

Typical dimensions: 768–4096 (model-dependent)

Strengths

- Handles **polysemy**
- Captures **compositional meaning** (“not good” ≠ “good”)
- Strong transfer across tasks (fine-tune or use as embeddings)

Limitations

- More compute/latency than static embeddings
- Long documents require **chunking + aggregation**
- Embedding space can be task-sensitive (may need finetuning)

Sentence embeddings

- **Sentence embeddings** are dense, fixed-length vectors that represent the **overall meaning** of a sentence in a continuous vector space.
- Typical size: **384–1536 dimensions**
- Similar sentences → **close vectors**
- Unrelated sentences → **far apart**

Two main approaches:

1. **Contextual model + pooling**
(baseline, simple)
2. **Dedicated sentence transformer models**
(state-of-the-art)

Word-level representations are not enough.

- Same word, different meaning:
 - “*bank*” (*finance*) vs “*bank*” (*river*)
- Averaging word vectors:
 - ignores **word order**
 - ignores **negation**
 - weak compositional meaning

Sentence embeddings exist to represent meaning at the sentence level directly.

Sentence Transformers (The Breakthrough)

Sentence-BERT (SBERT) and successors fix this.

How:

- Siamese / triplet architecture
- Contrastive loss on sentence pairs
- Explicitly trained for **cosine similarity**

Result:

- Similar sentences → pulled together
- Dissimilar sentences → pushed apart
- **This is the dominant approach in 2025–2026.**

Leading Models (2025–2026)

Open-source

- all-mpnet-base-v2
- all-MiniLM-L6-v2 (fast baseline)
- **E5 family** (instruction-tuned)
- **BGE series** (strong retrieval, multilingual)
- Snowflake Arctic, Nomic, UAE-large

Proprietary

- OpenAI text-embedding-3-large
- Cohere embed-v4
- Voyage-large
- Google Gemini embeddings

Document embeddings

- **Document embeddings** = dense, fixed-length vectors (**512–4096 dims**) for an **entire document**
- Encode **topic + themes + structure + intent**
- Similar documents → **close** (cosine / dot); different → **far**
- Where Document Embeddings Fit

Where Document Embeddings Fit

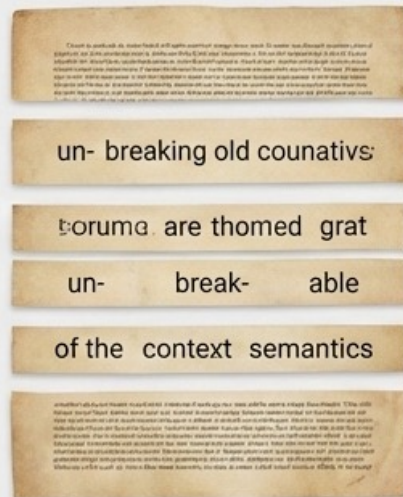
- **Word:** lexical meaning
- **Sentence:** compositional meaning (short text)
- **Document:** global topic + discourse + structure
- 2026 pattern: **query = sentence**, **corpus = document/chunks**

1. Document Level



Overall context & meaning.
High-level representation.

2. Sentence Level



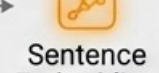
Sentence Embedding



Sentence Embedding



Sentence Embedding

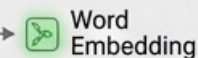


Sentence Embedding

Sentence meaning & structure.
Mid-level representation.

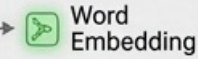
3. Word Level

un- break- book- able



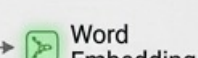
Word Embedding

standardking old counativor



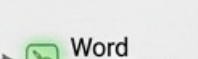
Word Embedding

has are honned grat



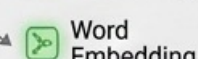
Word Embedding

un- break- able



Word Embedding

of the context semantics



Word Embedding

Individual word semantics.
Basic representation.

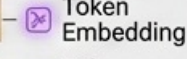
4. Token Level

un- break- break- able



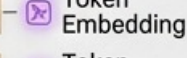
Token Embedding

un- break- break- able



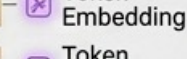
Token Embedding

un- break- break- able



Token Embedding

un- break- break- able



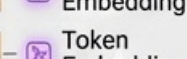
Token Embedding

un- break- break- able



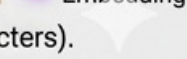
Token Embedding

un- break- break- able



Token Embedding

un- break- break- able



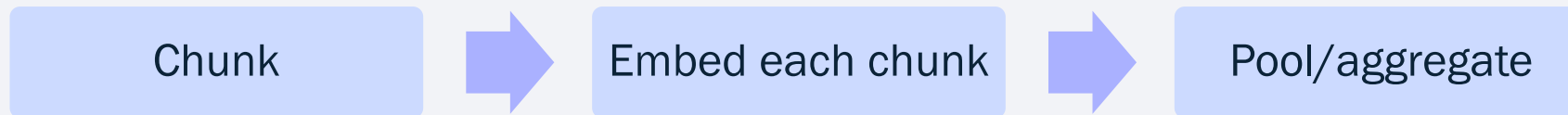
Token Embedding

Smallest units (sub-words, characters).
Fine-grained representation.

Level	Best for	Typical dims
Word	syntax, lexical similarity	100-300 (static)
Sentence	STS, short-query retrieval	384-1024
Document	RAG corpus retrieval, topic-level match	768-4096

How Document Embeddings Are Created

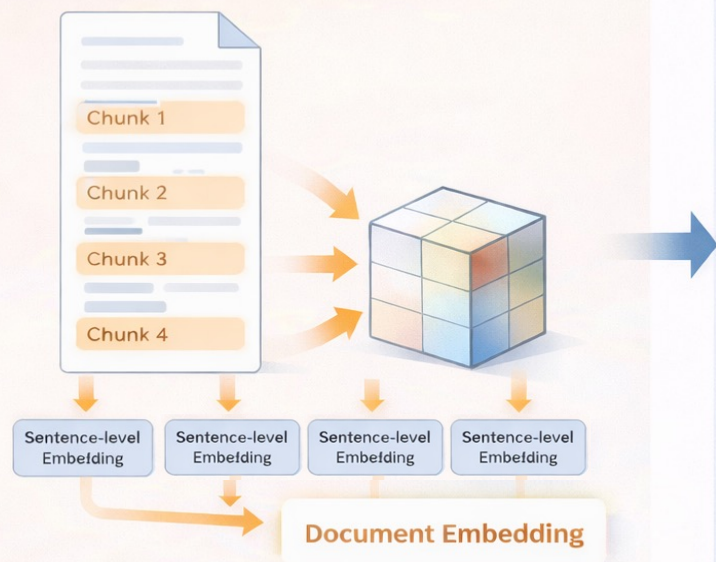
- Strategy A: Chunk + Aggregate (most common)



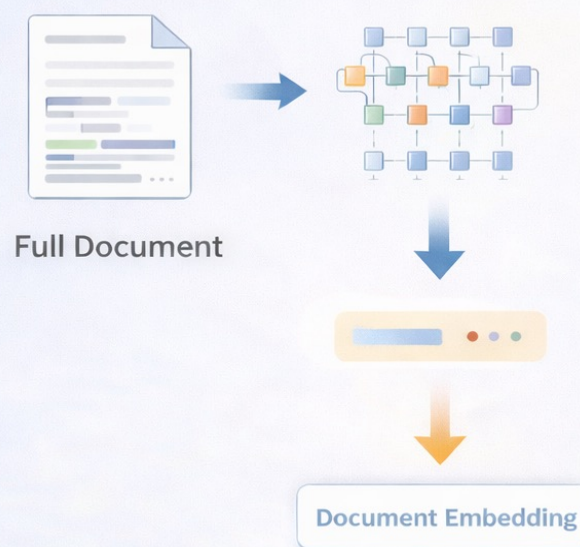
- Strategy B: Long-context model (increasingly dominant)



Strategy A: Chunk + Aggregate



Strategy B: Long-Context Model



Strategy A: Embed document chunks, then combine sentence-level embeddings.

Strategy B: Embed the whole document directly using a long-context model.

Strategy A:

Long-range dependencies are broken

- Chunk 1 cannot “see” Chunk 4

Structure is lost

- Section relationships disappear

Boundary artifacts

- Meaning split across chunks

Aggregation inevitably **throws information away**

Strategy B:

The model sees:

- discourse flow
- repeated themes
- section hierarchy
- cross-references

Meaning emerges globally, not chunk-by-chunk.

Models & When They Shine

- **General:** E5 / BGE / MPNet + aggregation
- **Long-context:** long-doc tuned embedding models
- **Domain-specific:** e.g., SPECTER for scientific papers
- **Multilingual:** multilingual E5 / BGE-M3

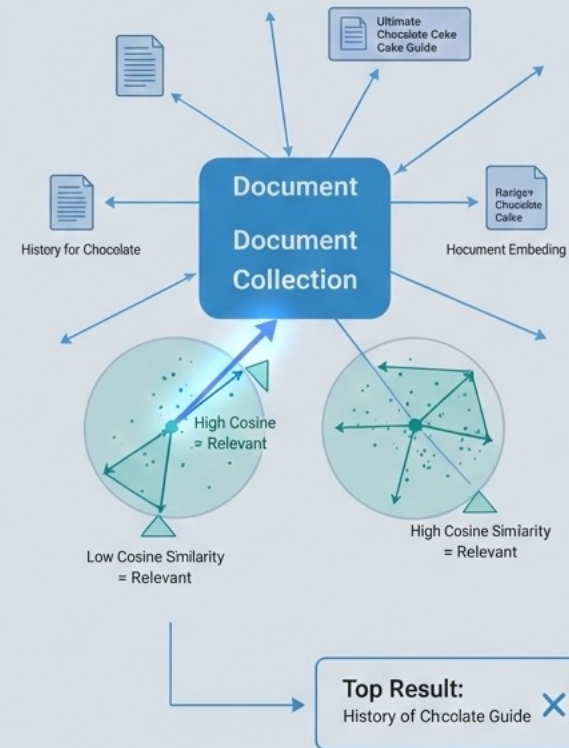
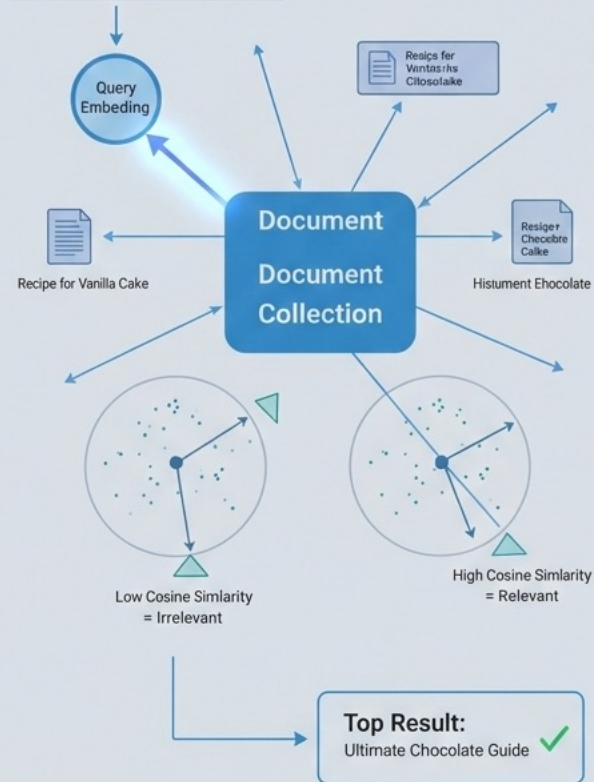
Key Properties of Good Document Embeddings

- **Topic robustness** (same topic, different wording)
- **Length invariance** (abstract vs full paper)
- **Asymmetric match** (short query ↔ long document)
- **Domain & multilingual support** (when needed)

Retrieval with Embeddings & Cosine Similarity

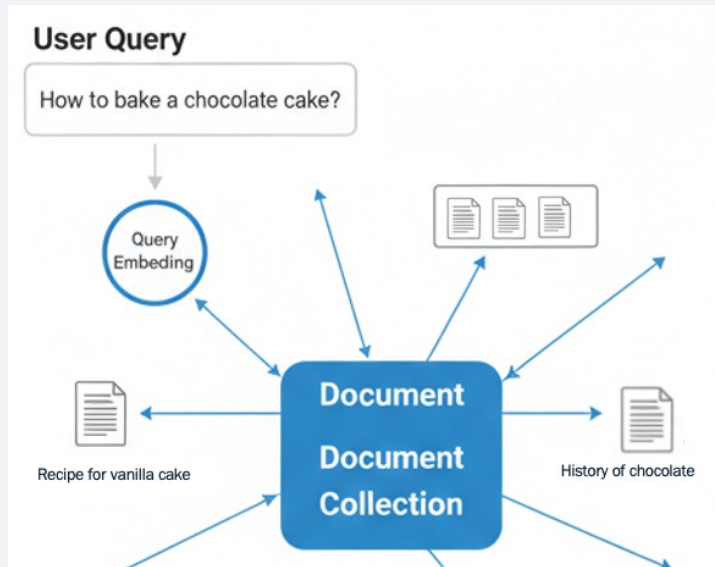
User Query

How to bake a chocolate cake?



Practical Implications for RAG / Retrieval

- Document embeddings power the **corpus index** in RAG
- **Chunking strategy** drives recall
- Dimensionality trade-off: **quality vs storage/latency**
- Best practice: **hybrid retrieval** (dense + BM25 + filters)



Retrieval:

- extracting potentially relevant items (usually documents, passages, web pages, records, images, products, etc.)
- from a large collection based on
- a user's information need, typically expressed as a **query**.

Corpus index

- The **searchable** structure that contains representations of the entire corpus, optimized for **retrieval**.

Multimodal embeddings

- **Modalities:** different types of data (text, images, audio, video, medical scans, diagrams, etc.)

The goal of multimodal models: Create a **shared embedding space** where:

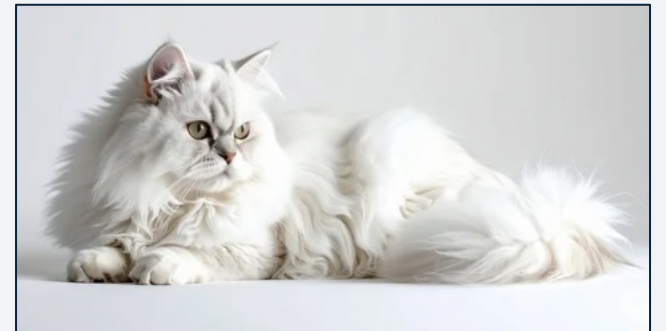
- Related things from **any modality** end up close together (e.g., photo of dog + word "dog" + barking sound clip).
- Unrelated things stay far apart.

Contrastive learning:

technique that teaches models to

- identify similarities and differences in data
- by grouping similar examples (positive pairs) together in a vector space while
- pushing dissimilar ones (negative pairs) apart.

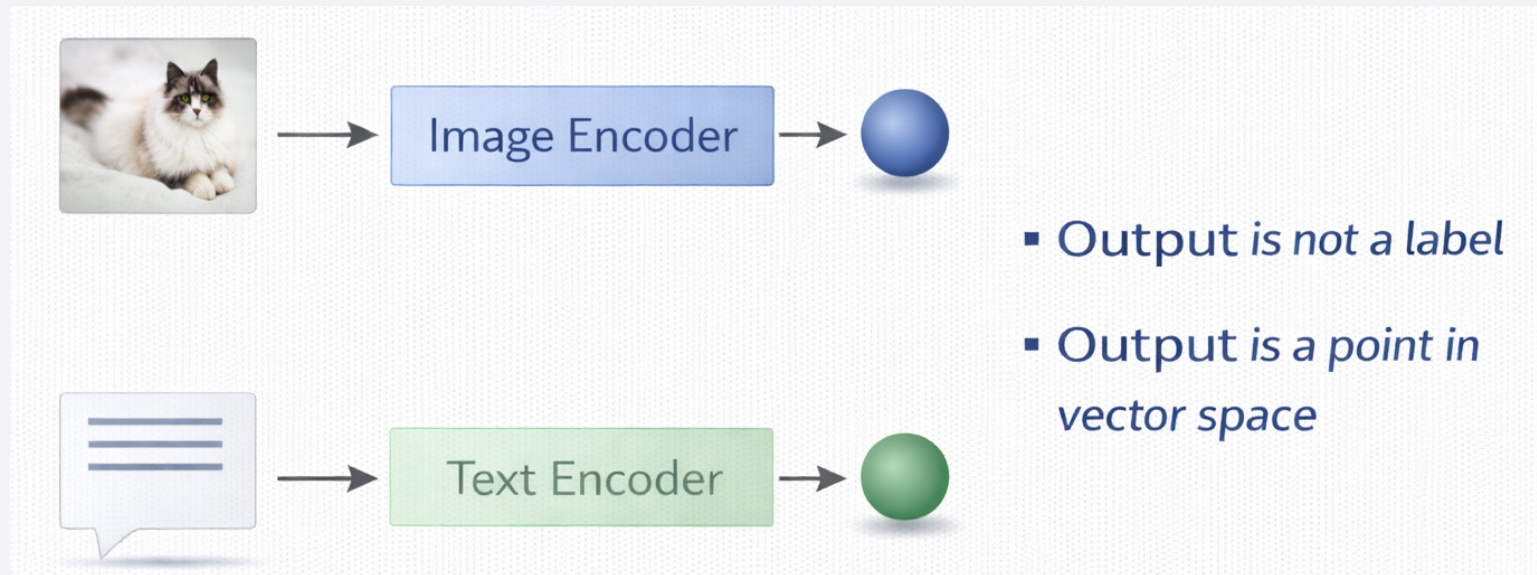
A golden retriever puppy playing with a blue ball



A fluffy cat lying down with tail wrapped around

Multimodal embeddings: contrastive learning

- Learn embeddings by pulling matching pairs together
- Pushing non-matching pairs apart
- Distance in vector space is trained to reflect semantic correspondence.



Contrastive Learning

Positive pairs

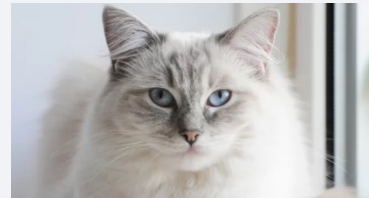
- Represent the **same underlying concept**
- Should be **close** in embedding space

Negative pairs

- Represent **different concepts**
- Should be **far apart**

Intra-modality contrastive learning:
Learning structure within the same modality

A fluffy Ragdoll cat lying down with tail wrapped around



Positive intra-modality

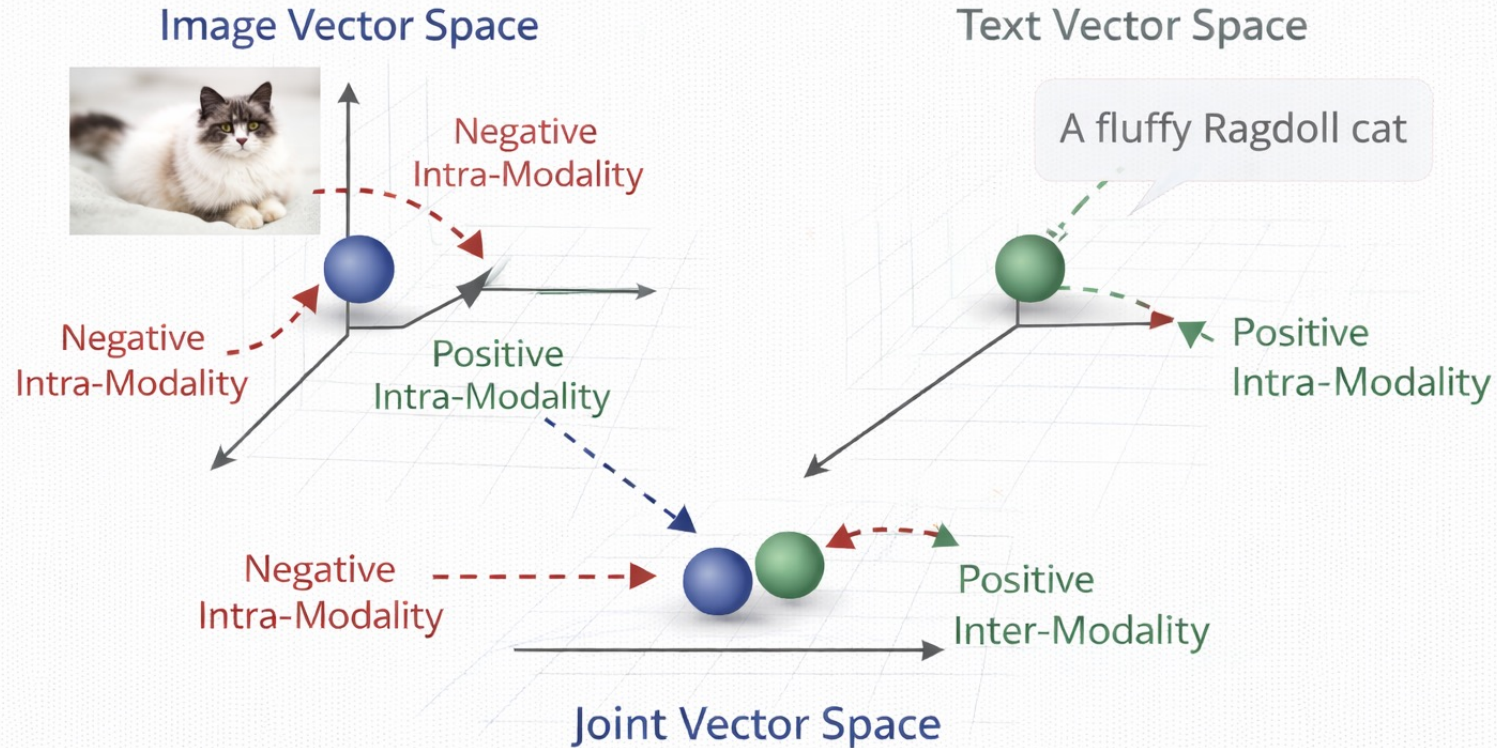
- Image $\leftrightarrow$ image (same object, different views)
- Text $\leftrightarrow$ text (paraphrases)

Negative intra-modality

- Image $\leftrightarrow$ image (different objects)
- Text $\leftrightarrow$ text (unrelated meanings)

Multimodal Vector Space

Combining image and text into a joint vector space



	Positive (pull together)	Negative (push apart)
Intra-modality	Same meaning, same modality	Different meaning, same modality
Inter-modality	Same meaning, different modalities	Different meaning, different modalities

Structured data embeddings

Structured data is data organized according to a **fixed schema**:

- Rows = entities (customers, transactions, products)
- Columns = predefined fields with known types (number, category, boolean, date, ID)

A **structured data embedding** is a numerical vector that represents:

- the **entire row (or entity)**, considering **all fields together**

This allows:

- similarity search over rows
- clustering entities
- anomaly detection
- retrieval or ranking of records

customer_id	age	country	account_balance	active
8123	34	AU	12500	TRUE

Instead of treating:

- age = 34
- country = AU
- balance = 12,500

as separate values, we map the whole record into **one vector**: A **structured data embedding** is a numerical vector that represents:

- the **entire row (or entity)**, considering **all fields together**
- So instead of treating:
 - age = 34
 - country = AU
 - balance = 12,500
- as separate values, we map the whole record into **one vector**:

Why structured data embeddings are hard

Why structured data embeddings are hard

- No universal meaning
- Schema-dependent semantics
- Mixed data types

What structured embeddings are used for

- Entity similarity (find similar customers)
- Recommendation (similar products/users)
- Fraud & anomaly detection
- Ranking and retrieval over records
- Downstream ML tasks

How structured data embeddings are built

Step 1: Encode each column

Numbers → normalized (z-score, min-max)

Categories → learned embeddings or one-hot

Booleans → binary

Dates → cyclic or relative encodings

Step 2: Combine features

Concatenate encoded fields

Or pass through a neural network

Or use attention across fields

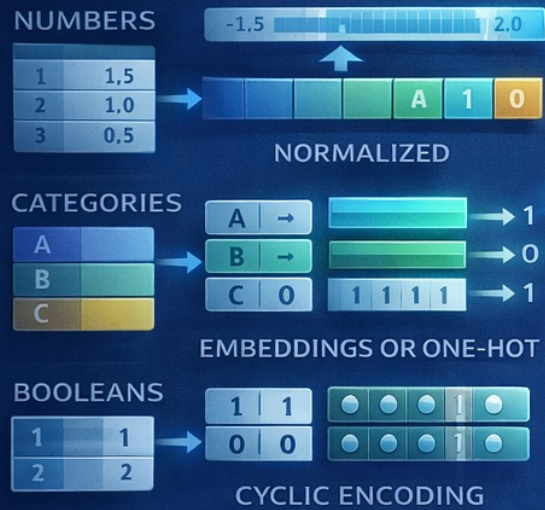
Step 3: Learn a joint embedding

Train the model so similar rows map close together

Supervision depends on task: same customer type, same risk profile, similar behavior

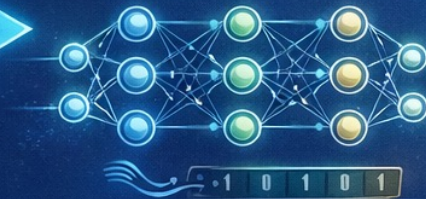
HOW STRUCTURED DATA EMBEDDINGS ARE BUILT

STEP 1: ENCODE EACH COLUMN

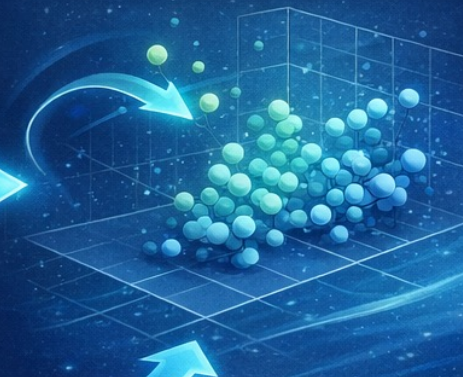


STEP 2: COMBINE FEATURES

Concatenate encoded fields
Or pass through a neural network
Or use attention across fields



STEP 3: LEARN A JOINT EMBEDDING



Train the model so similar rows map close together

Evaluation: Measuring quality

- Embeddings are not outputs — they are **representations**.
- There is **no universal “good embedding”** — quality is **task-dependent**.
- An embedding is **good** if **geometric relationships in vector space align with the task-relevant notion of similarity**.
- Embeddings turn data into points in a vector space, where:

distance $\approx$ similarity

geometry $\approx$ meaning

But:

- this meaning is **learned**
- it is **model-dependent**
- and often **invisible**

Two embeddings can look numerically valid but:

- retrieve the wrong items
- rank relevant items poorly
- fail under domain shift

Embeddings often:

- “look” reasonable
- cluster nicely in plots
- pass small demos

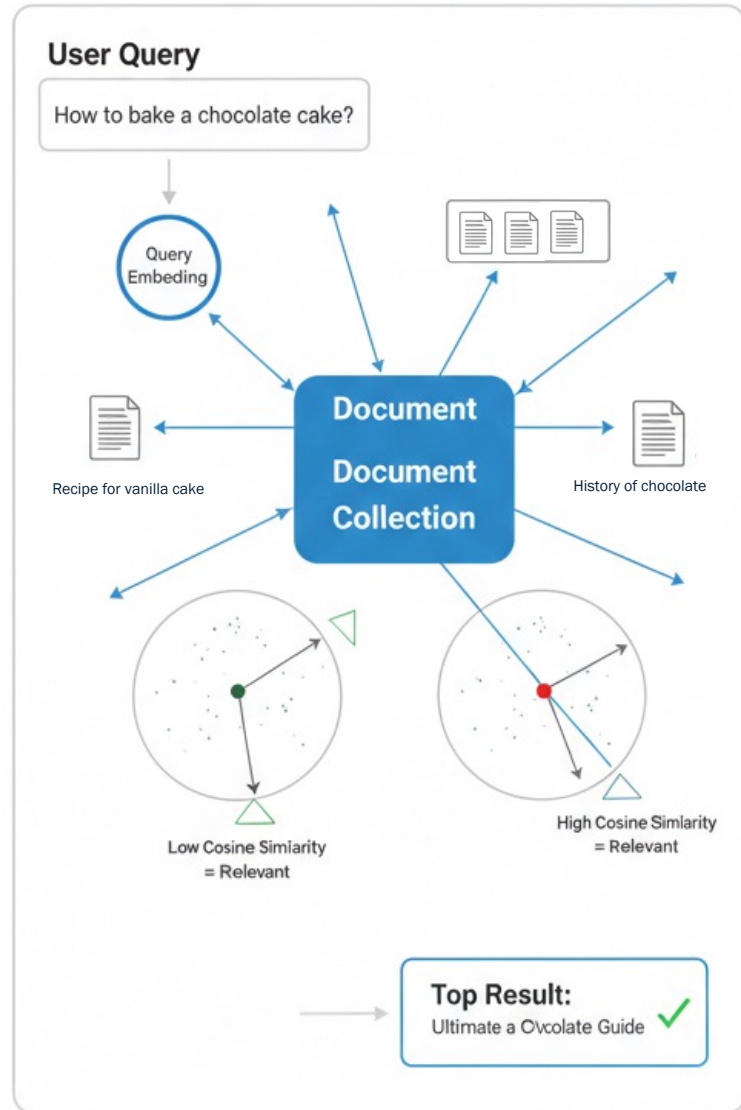
Yet fail in practice because:

- small distance errors compound at scale
- ranking matters more than raw similarity
- edge cases dominate user experience

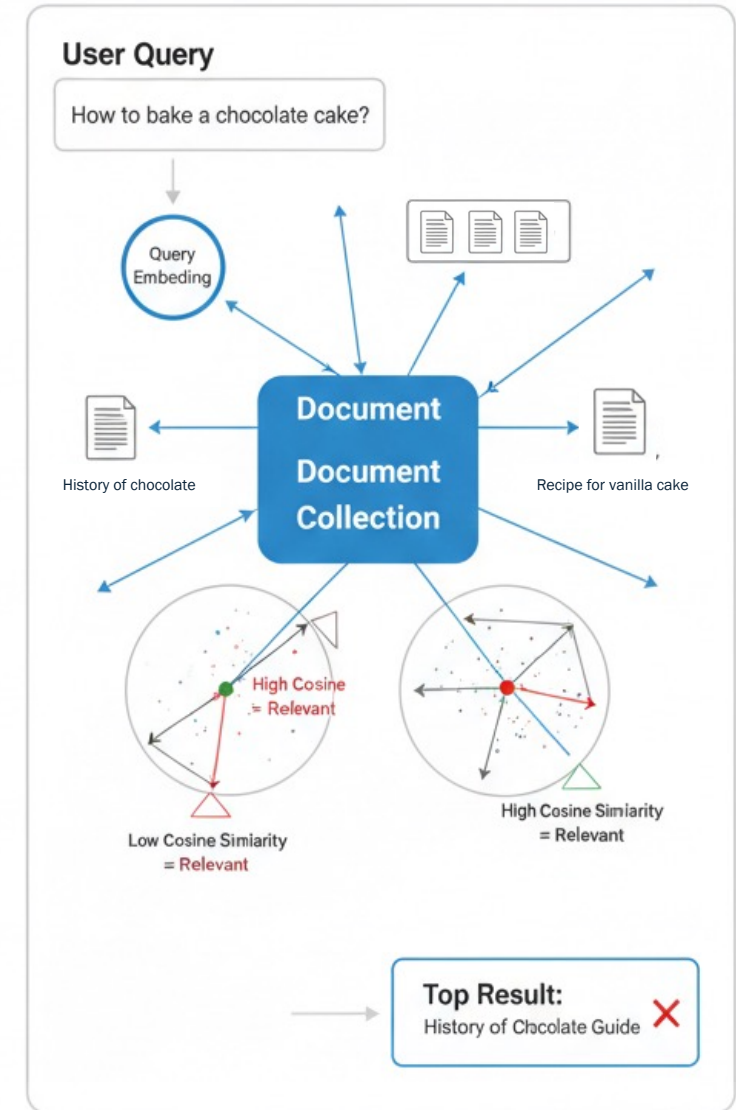
Visual intuition does not predict retrieval quality.

Retrieval

1. Good Quality Embeddings



2. Bad Quality Embeddings



Measuring Quality

- Same vocabulary
- Same inputs
- Different geometry

Because the **objective changes**

- There is no single “true” semantic space.

Cold

biomedical:
near "influenza",
"rhinovirus", "hypothermia"

general:
near "hot", "winter", "cool"

News

legal:
near "judgment",
"precedent", "litigation"

general:
near "box", "example",
"situation"

Retrieval metrics

- Retrieval metrics apply to **any modality where retrieval = ranking items by similarity**
- What changes is **how relevance is defined**, not the metrics themselves.



Retrieval metrics

Recall@K

Did we retrieve the right things at all?

Of all the relevant items that exist, how many did we retrieve in the top K?

Works for:

Text, images, multimodal

measures **coverage**, not order.

If low, embeddings/ indexing are broken

nDCG@K

Did we rank the best items first?

How good is the **ordering**, with higher reward for putting more relevant items earlier?

Unlike recall: relevance can be **graded**, not just yes/no

Works for:

Text, images, multimodal

Multimodal: strong vs weak cross-modal alignment

measures **ranking quality**, not just retrieval success.

MRR

How fast do we hit the first correct result?"

Reciprocal of the rank of the first relevant item.

Works for:

Text, images, multimodal

Use when **one correct item** matters most

Less useful when many relevant items exist

Benchmarks

Benchmarks provide **standardized datasets + metrics** so that:

- different embedding models can be compared fairly
- improvements are measurable
- regressions are detectable

They test embeddings **indirectly**, via the tasks embeddings are used for.

Spaces mteb/leaderboard like 7.02k Running on CPU UPGRADE

General Purpose < Embedding Leaderboard

Multilingual

English

Human Benchmark

Image

Domain-Specific

Language-specific

Miscellaneous

Retrieval

RTEB Multilingual

RTEB English

Image

VideoRe V3

Jina Visual Document

This leaderboard compares 100+ text and image embedding models across 1000+ languages. We refer to the publication of details on metrics, languages, tasks, and task types. Anyone is welcome to [add a model](#), [add benchmarks](#), [help us annotations](#) or [propose other changes to the leaderboard](#).

MTEB(Multilingual, v2)

A large-scale multilingual expansion of MTEB, driven mainly by highly-curated community contributions covering 250+ languages.

Cite and share this benchmark

Customize this Benchmark

Advanced Model Filters

Click for More Info

Summary Performance per Model Size Performance per Task Type Performance per task Performance per language

Rank (Bo..	Model	Zero-shot	Memory Us..	Number of P..	Embedding D..	Max Tokens	Mean
1	KallM-Embedding-Gemma3-12B-2511	73%	44884	11.8	3840	32768	72..
2	Llama-embed-nemotron-8b	99%	28629	7.5	4096	32768	69..
3	Qwen3-Embedding-8B	99%	14433	7.6	4096	32768	70..

MTEB — Massive Text Embedding Benchmark
BEIR — Benchmark for Information Retrieval
STS Benchmarks (STS-B, STS12–16)
Multimodal Retrieval Benchmarks
Production / Custom Benchmarks

Where to find:

Hugging Face (primary hub)

- Central repository for models and evaluation datasets
- De-facto distribution channel for embedding benchmarks

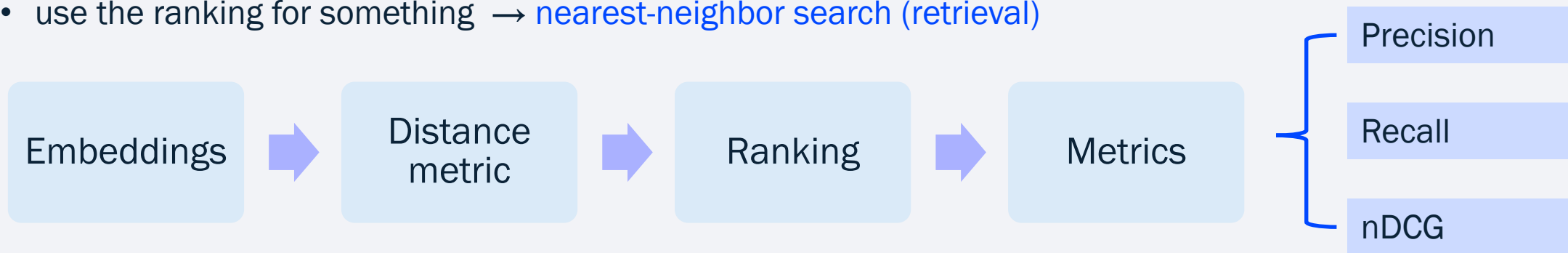
<https://huggingface.co/spaces/mteb/leaderboard>

Benchmark	Focus	Why Used	
MTEB	General embedding quality	Broad, modern standard	Covers many downstream tasks , not just retrieval Evaluates sentence & document embeddings Includes: Retrieval, Semantic Text Similarity (STS), Clustering, Classification, Reranking
BEIR	Retrieval robustness	Zero-shot generalization	Tests zero-shot retrieval across many domains No task-specific fine-tuning allowed Emphasizes generalization
STS	Sentence similarity	Interpretability	Sentence pairs labeled with graded similarity Measures how well cosine similarity matches human judgment
WordSim / Analogies	Word semantics	Legacy / analysis	
Multimodal sets	Cross-modal retrieval	Image-text alignment	MS-COCO Flickr30k LAION evaluation subsets
Custom evals	Real-world performance	Production truth	Built from real queries, real documents, real relevance judgments

How Embeddings Turn into Retrieval Metrics

A vector has **no meaning** until you:

- compare it to other vectors
- rank those comparisons
- use the ranking for something → [nearest-neighbor search \(retrieval\)](#)



Embeddings are not scored. Retrieval behaviour induced by embeddings is scored.

Did I return junk? → Precision
Did I miss good stuff? → Recall
Did I order the good stuff correctly? → nDCG

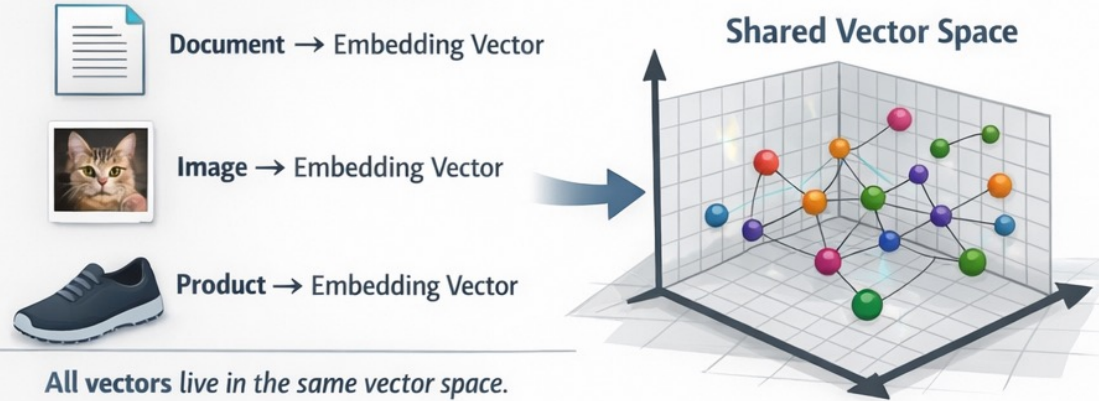
Beyond Keywords – What is Vector Search?

The Problem: Traditional keyword search relies on explicit matching (e.g., searching "cappuccino" won't find "latte").

- **The Solution:** Vector Search uses **semantic representations** of data.
- **How it Works:** It embeds the incoming query into the same vector space as the indexed items.
- **Versatility:** Works across various data modalities, including **text, images, and video**.

How Vector Search Works (Step by Step)

1. Represent Items as Vectors



2. Represent Query as Vector



3. Compare Vectors by Distance

- Cosine Similarity

- Dot Product

- Euclidean Distance

Closer → More Similar

Farther → Less Similar



4. Retrieve Nearest Neighbors

Top Results



1. Cozy Sneakers

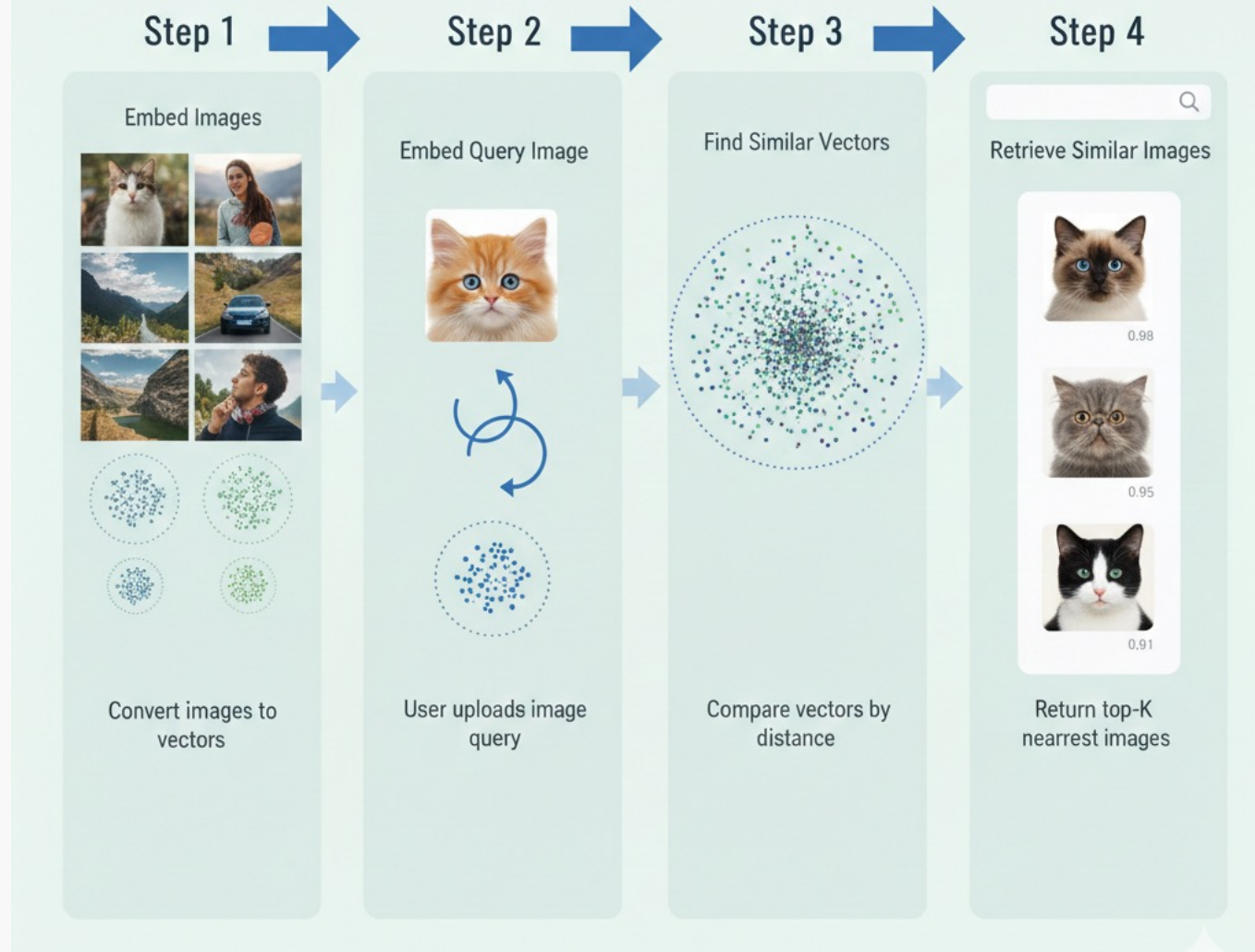


2. Soft Slippers



3. Casual Shoes

VECTOR SEARCH FOR IMAGES



Calculating Similarity – The Mathematics of Search

Euclidean Distance

“How far apart are these two points in space?”

Measures **straight-line distance**

Smaller = more similar

Range: 0 to ∞

When used:

Image embeddings

Clustering

Some legacy systems

$$\text{dist}(q, x) = \sqrt{\sum_{i=1}^n (q_i - x_i)^2}$$

Cosine Similarity (most common)

“Do these vectors point in the same semantic direction?”

Measures the **angle** between vectors

Ignores magnitude (length)

Focuses on **direction = meaning**

Range: -1 to 1 (in practice: 0 to 1)

When used

Text embeddings

Sentence / document retrieval

Most modern vector databases (default)

$$\text{cosine}(q, x) = \frac{q \cdot x}{\|q\| \|x\|}$$

Dot Product (fast, common)

Combines **direction + magnitude**

Bigger vectors can score higher

Range

Unbounded

When used

When embeddings are **already normalized**

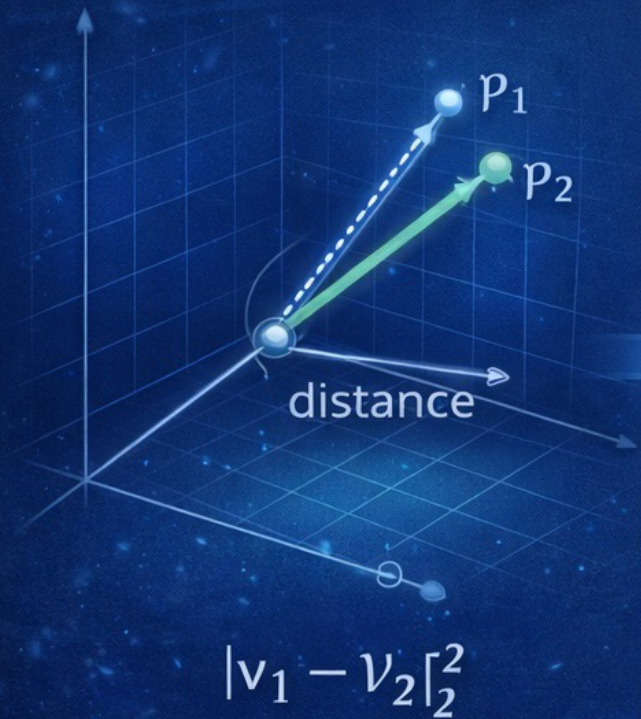
Large-scale retrieval systems

Learned ranking systems

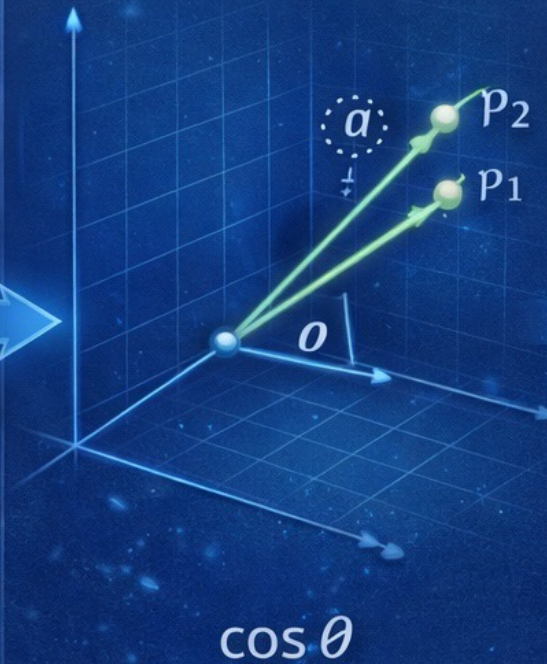
$$\text{dot}(q, x) = q \cdot x = \sum_{i=1}^n q_i x_i$$

VECTOR SEARCH: DIFFERENT APPROACHES TO CALCULATE SIMILARITY

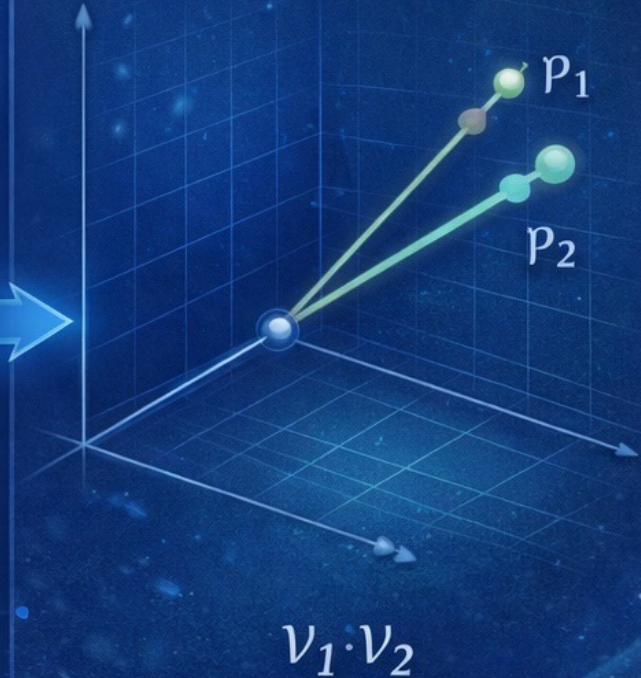
EUCLIDEAN DISTANCE (L2)



COSINE SIMILARITY



INNER / DOT PRODUCT



Why and How Vector Search Scales

Linear Search ($O(N)$)

- For each query vector, you compare it against **every stored vector**.
- Works for small datasets.
- Becomes **prohibitively slow** when you have millions or billions of embeddings (e.g., RAG, recommender systems).

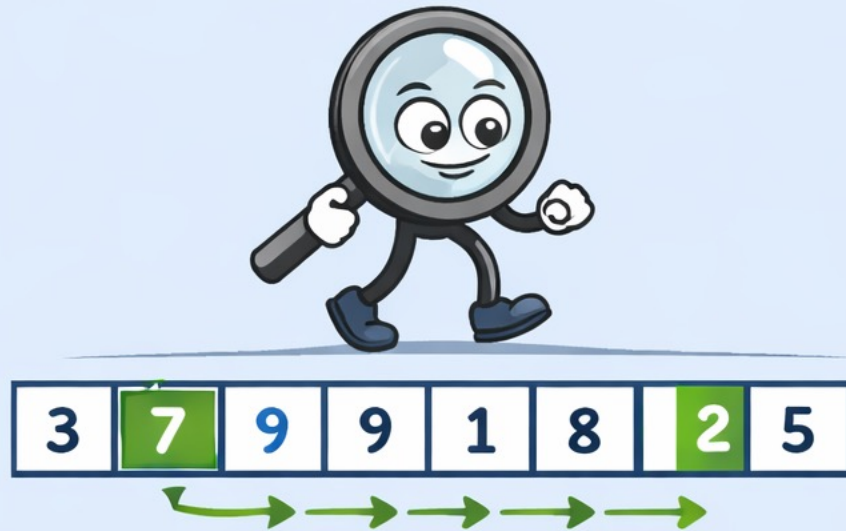
Approximate Nearest Neighbor (ANN) Search ($\sim O(\log N)$ or sub-linear)

- Instead of checking all vectors, ANN uses **indexes** (graphs, trees, quantization).
- It finds vectors that are *very close* to the true nearest neighbors, but not guaranteed exact.
- You trade **perfect accuracy** for **huge speed gains**.

Balance:

- **Scale** (how many vectors you can store),
- **Indexing time / memory** (cost to build and store the index),
- **Query performance** (latency vs recall).

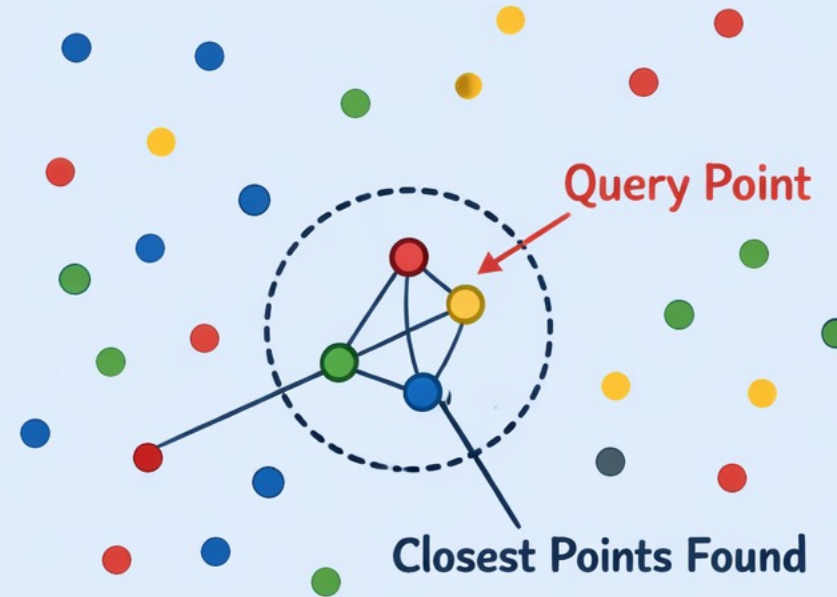
Linear Search ($O(N)$)



Checks Each Item One by One

Time: Slow ($O(N)$)

Approximate Nearest Neighbor (ANN) Search



Quickly Finds Nearest Points

Time: Fast ($O(\log N)$)

Key Search Algorithms

LSH (Locality Sensitive Hashing)

Similar vectors → same hash bucket

You hash vectors so that close ones collide.

At query time, you only search within a few buckets.

Why it exists

One of the earliest ANN methods.

Simple, mathematically elegant.

Trade-offs

Fast

Lower accuracy at high dimensions

Less popular in modern vector DBs

HNSW (Hierarchical Navigable Small Worlds)

Build a graph of vectors

Vectors are nodes; edges connect nearby vectors.

Search “zooms in” from coarse layers to fine layers.

Why it's dominant

Very high recall

Excellent latency

Works well in high dimensions

Reality

This is what **most modern vector databases use by default.**

ScaNN (Scalable Approximate Nearest Neighbor)

Partition space + compress vectors

Uses clustering + quantization to reduce computation.

Optimized heavily for hardware (SIMD/TPUs).

Why it matters

Designed by Google

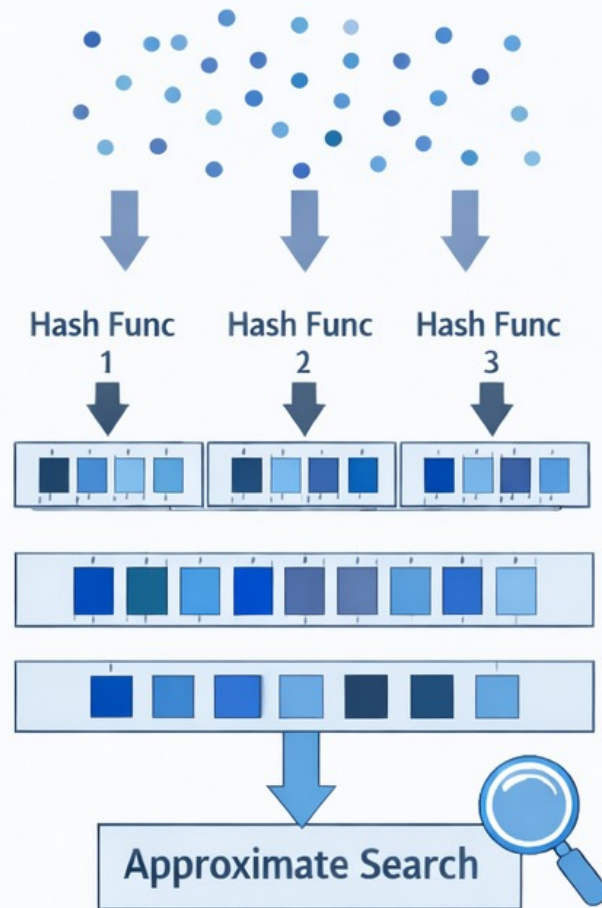
Extremely fast for large-scale production systems

Reality

Common in Google infra, less common as a standalone DB.

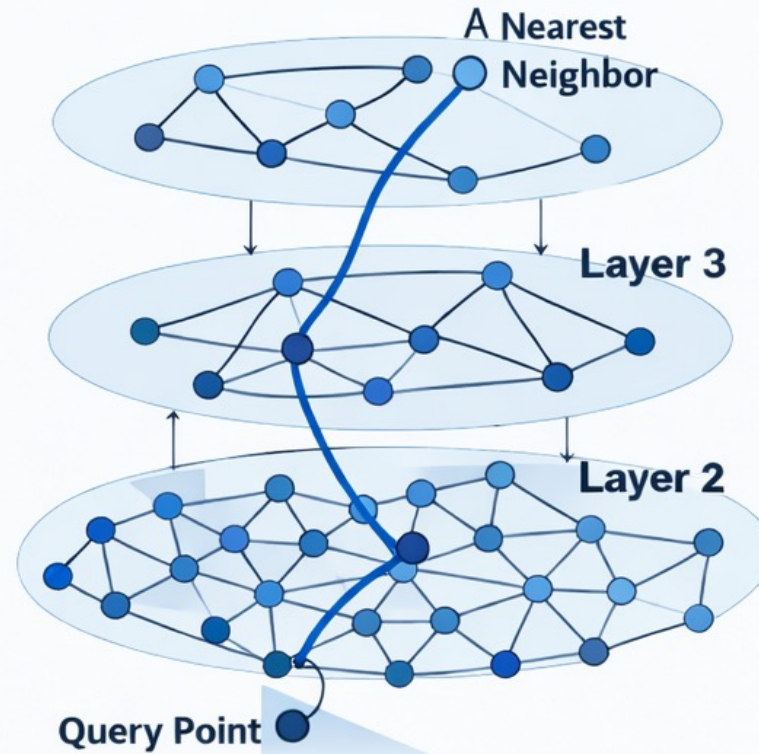
LSH

(Locality Sensitive Hashing)



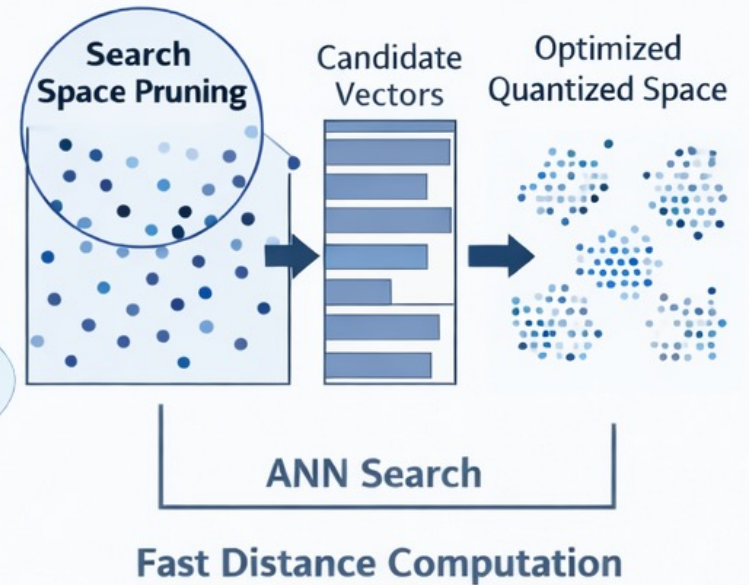
HNSW

(Hierarchical Navigable Small Worlds)

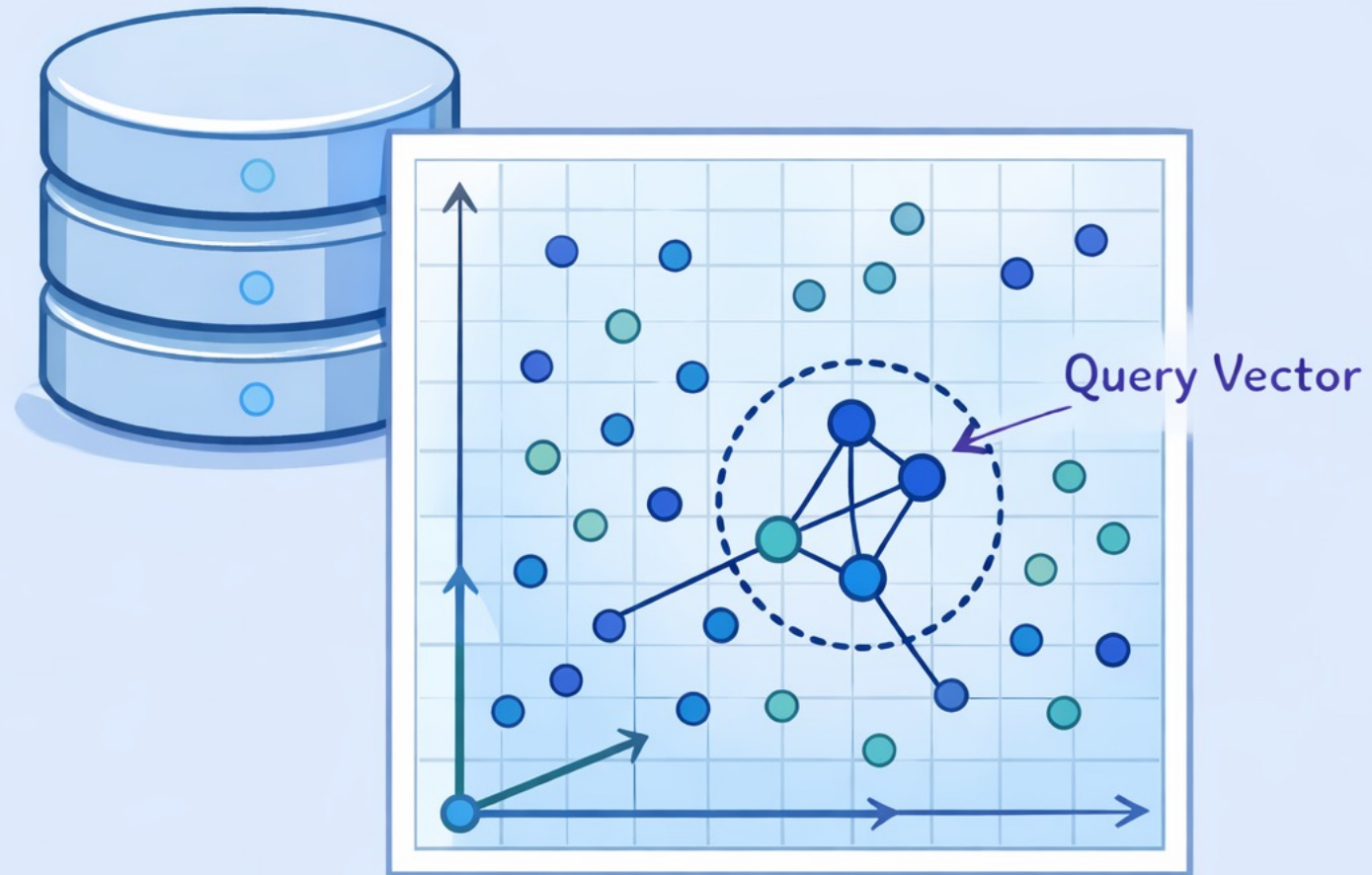


ScaNN

(Scalable Approximate Nearest Neighbor)



Vector Database



Vector Databases: What & Why

- **Vector databases** store and search data based on **meaning and similarity**, not exact matches.
- They became critical because:
- AI models convert text, images, audio into **vectors (embeddings)**
- Modern applications need to find **similar things**, not identical strings



Traditional Databases (What We Had)

Traditional databases (SQL, tables):

- Rows & columns
- Exact matches, filters, ranges

Excellent for:

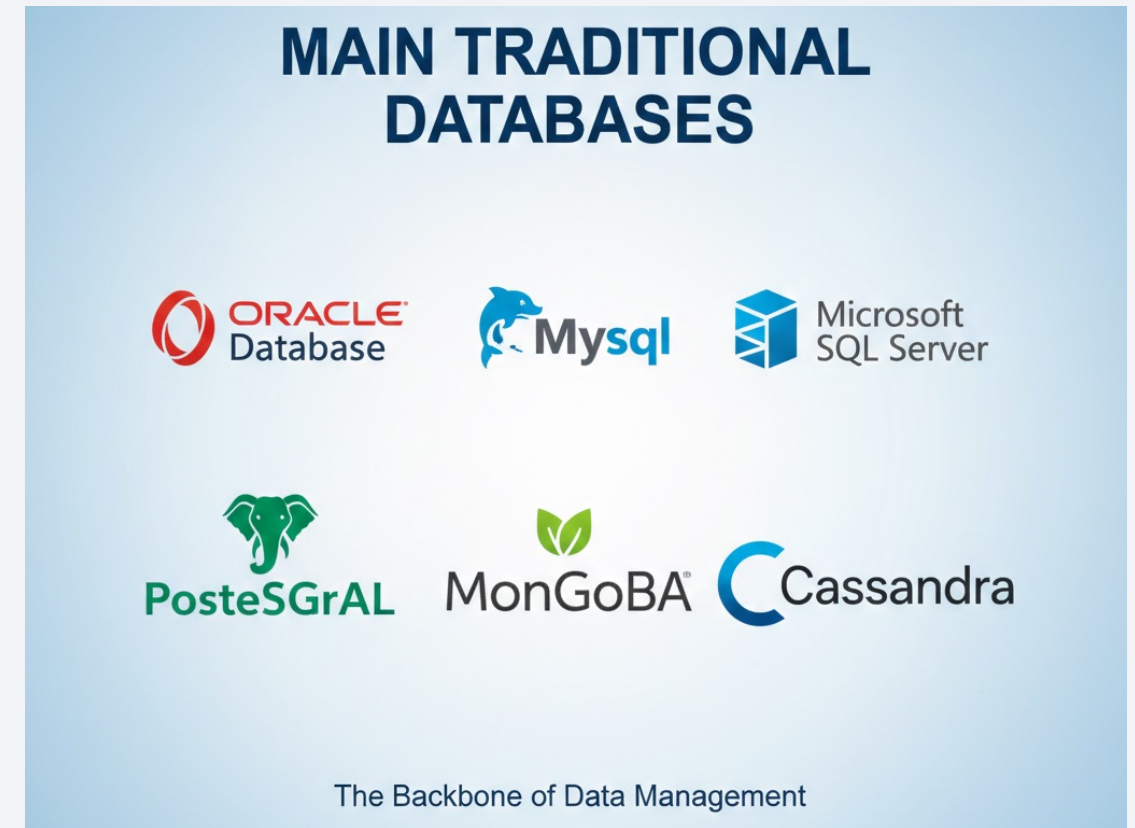
- transactions
- numbers
- categories

Terrible at:

- semantic similarity
- fuzzy meaning
- images, text understanding

Example queries:

- age > 30 AND city = Melbourne
- name LIKE '%iPhone%'



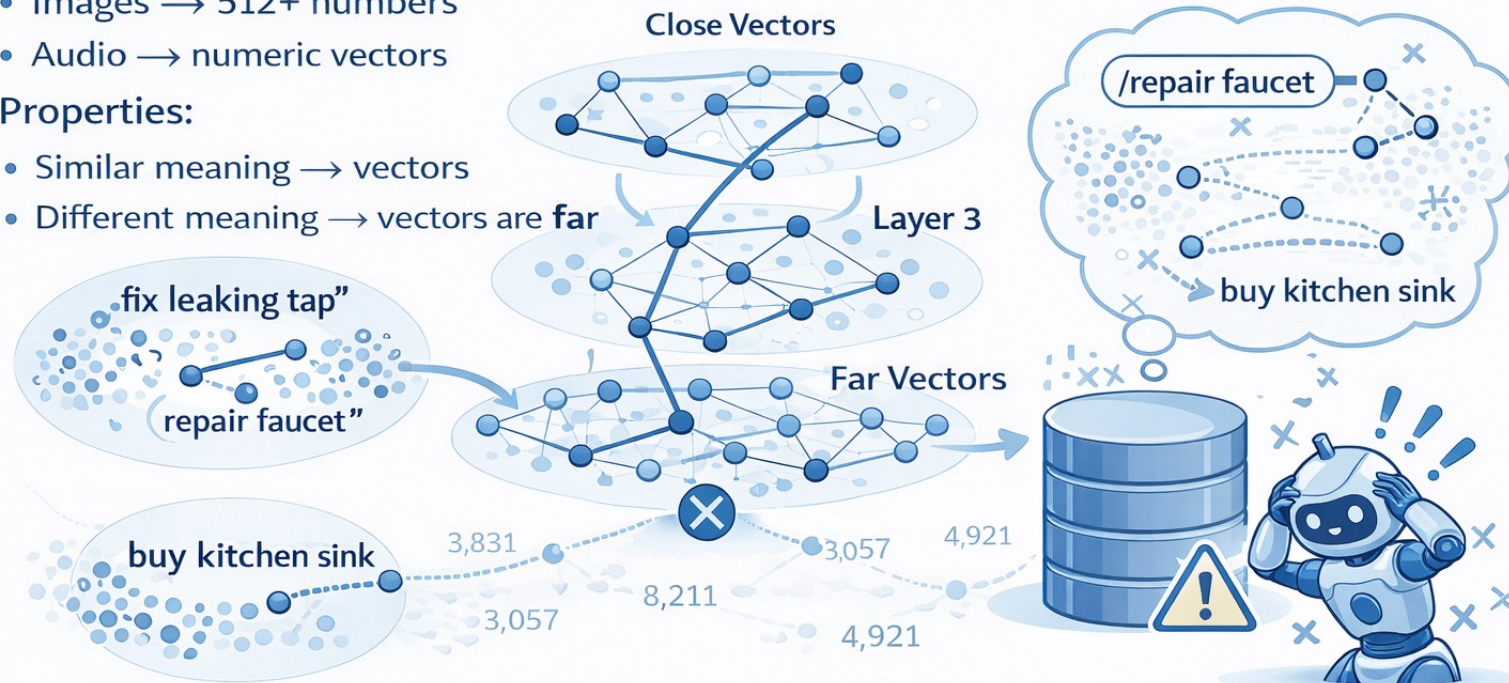
The Problem AI Introduced

Modern AI turns data into vectors:

- Text → 768 numbers
- Images → 512+ numbers
- Audio → numeric vectors

Properties:

- Similar meaning → vectors
- Different meaning → vectors are far



What Is a Vector Database?

A **vector database**:

- Stores vectors efficiently
- Finds the **nearest vectors** to a query vector
- Returns results ranked by similarity
- Stores metadata alongside vectors

Core capability:

- **Similarity search / nearest-neighbor search**

Instead of:

“WHERE text = ...”

You ask:

“Find the 10 most similar vectors”

Common real-world uses:

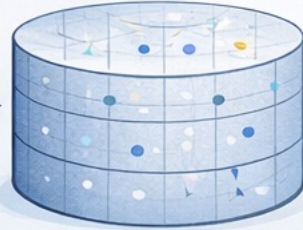
- **Chat with documents (RAG)**
- Semantic search
- Recommendations
- Image / video similarity
- Customer support automation
- Anomaly & fraud pattern search

All of these rely on **fast similarity search at scale**.

Vector Database Workflow

1 Prepare Data and Query

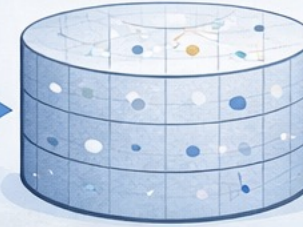
Embeddings (query)
{ vector: [0.12, 0.65, ...5.57]
 metadata: { _____
 } ...



{ vector: [0.11, 0.27, 5.72],
 metadata: { _____
 •••• "Your purchase is eligible for a
 } full refund...",...
 } ...

3 Query Vector and Retrieve

Embeddings (query)
{ vector: [0.12, 0.65, ...5.57]
 metadata: { _____
 } ...



{ vector: [0.11, 0.27, 5.72],
 metadata: { _____
 •••• "Your purchase is eligible for a
 } full refund...",...
 } ...

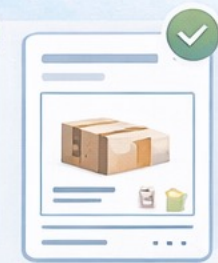
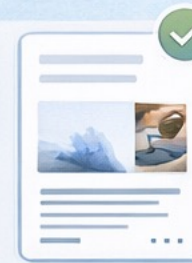
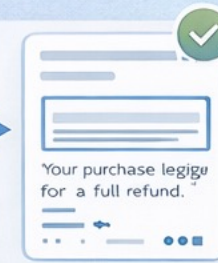
3 Query Vector and Retrieve



Can I return this
item I purchased?



Embeddings (query)
{ vector: [0.11, 0.27, ...5.72]
 metadata: { _____
 } ...



Hybrid Search & Advanced Filtering

Vector databases are great for

- Meaning, intent, paraphrases
- Fuzzy or abstract queries

But **weak** for:

- Exact IDs (INV-2024-0931)
- Codes, SKUs, error numbers
- Highly domain-specific jargon
- Legal / financial literal terms

Hybrid search exists to fix this.

Hybrid Search = Dense + Sparse retrieval

- Combine vector similarity with keyword-based matching, then merge or rerank the results.

Two retrieval signals

1. Dense (semantic)

1. Vector similarity (cosine / dot product)
2. Captures *meaning*

2. Sparse (lexical)

1. Keyword / token overlap (BM25-style)
2. Captures *exact terms*

Hybrid Search in Vector Databases



Filtering capabilities

Pre-filtering (before vector search)

Shrink the haystack before looking for the needle.

What happens

Apply metadata constraints **before** similarity search

Vector index is queried only on a subset

Faster queries

Higher precision

Lower false positives

Essential for compliance and multi-tenant systems

Examples

```
document_type = "contract"  
region = "EU"  
date > 2022  
customer_id = 123
```

Post-filtering (after retrieval)

First retrieve broadly, then judge carefully.

What happens

Run a fast, approximate vector search (ANN)

Take top-K candidates

Apply:

- Rerankers

- Cross-encoders

- Business rules

- Hybrid score fusion

ANN search is optimized for **speed**, not perfect ranking

Expensive models are applied only to a small candidate set



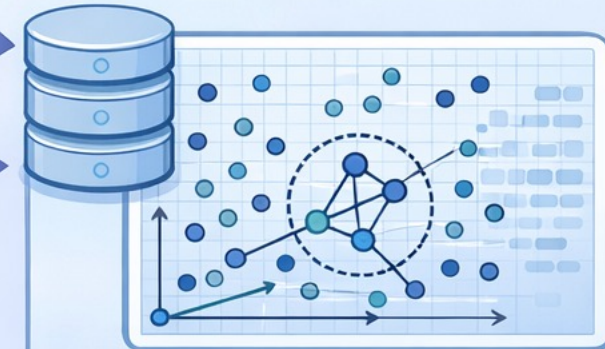
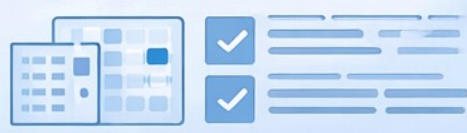
Image Query

Keyword Search (BM25 / Sparse)

mountain lake forest



Pre-filter
(metadata
constraints)



ANN Vector Search

Merge Dense + Sparse Results

Final Ranked
Results

1



2



3



Operational Realities (Mutation & Maintenance)

Embeddings are **model-dependent projections**.
If the model changes, the vector space changes.

Even small updates can cause:

- Rotation of the vector space
- Scaling differences
- Reordered neighborhoods
- Broken similarity relationships

So:

- Old vectors + new query embeddings → **nonsense distances**
- Similarity scores become meaningless

This is **not data drift** — it is **representation drift**.

Embedding Mutation: Embeddings can drift or change when underlying models are retrained or updated.

Version Consistency: It is critical to ensure that the model used for inference matches the model used for training/indexing.

Management Tasks: Need automated processes to store, manage, and purge old embeddings to stay within budget

Operationally, vector databases demand

- strict embedding version control,
- automated reindexing, and
- lifecycle management—

otherwise similarity search silently degrades and costs explode.

Example: Pinecone Vector Database

Pinecone is a **managed vector database**.

Think of it as a **filing system**:

- **Organization** → entire company
- **Project** → one application or environment
- **Index** → one searchable vector collection
- **Namespace** → logical partition inside an index
- **Record** → one vector + metadata
- This hierarchy keeps data isolated, scalable, and secure.

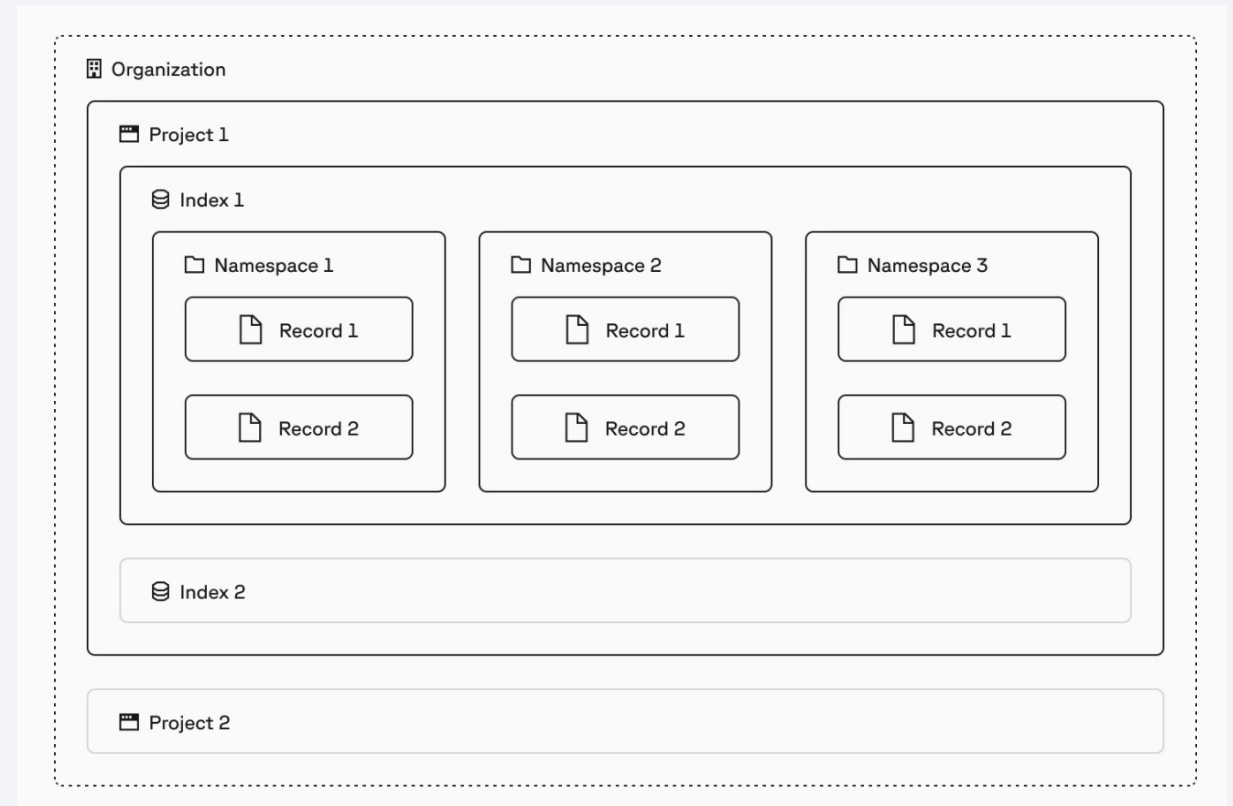


Pinecone

- **Account / Cluster / Workspace** ($\approx$ Organization) $\rightarrow$ billing & users
- **Index / Collection** $\rightarrow$ the searchable vector set (the most important concept)
- (Optional) **Partition / Tenant / Namespace / Shard** $\rightarrow$ logical isolation inside index (for multitenancy or performance)
- **Record / Vector / Point** $\rightarrow$ individual entry

Namespaces enable:

- multi-tenant apps
- dev / prod separation
- language or customer isolation





Practice